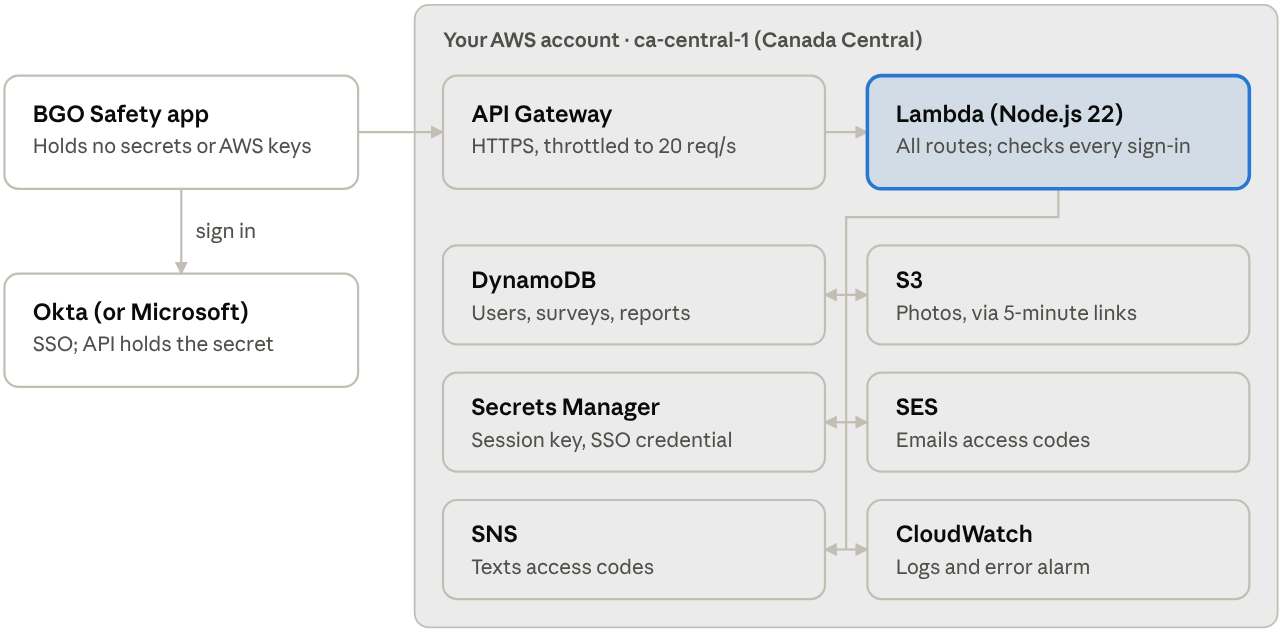
# BGO Safety — AWS Serverless Setup Guide

Nov 4, 2026 · @William Ren

## Overview

One `sam deploy` builds the whole backend in **ca-central-1**, so property and incident data stays in Canada. Emails and texts go out through your existing SES and SNS setup in us-east-1. There are no servers to run: AWS runs the API code only when the app calls it, and you pay per request.

**The app talks only to the API; secrets and data stay in AWS**




BGO Safety on AWS · app, API, 6 managed services

The app holds only public settings (the API address and whether to show the SSO button); Okta or Microsoft settings live in the API. Access codes are created, sent and checked in AWS; the SSO credential and session signing key live in Secrets Manager; photos go straight to S3 through short-lived links the API hands out for properties the user can access. The rest of this guide follows the order you set it up: email and text delivery (sections 1–2), SSO (3), deploy (4–5), data (6), the app (7), then reference and operations (8–9).

# Configuration checklist

You run `sam deploy` **once**. Everything it needs is gathered first (steps 1–6, all known in advance because the address is `https://bgosafety.refineddata.com`). The steps after it need no second deploy.

| # | When | What | Value to enter / action | Where | Status |
|---|---|---|---|---|---|
| 1 | Before | AWS account, region | Deploy into `ca-central-1`; CLI signed in | `aws configure sso` | To do |
| 2 | Before | SES (check only) | `no-reply@refineddata.com` verified with DKIM and out of the sandbox in **us-east-1** | SES console, us-east-1 | To do |
| 3 | Before | SNS (check only) | Out of the SMS sandbox, spend limit set, in **us-east-1** | SNS console, us-east-1 | To do |
| 4 | Before | Certificate ARN | Your `*.refineddata.com` certificate in **ca-central-1** (section 4) | ACM console, ca-central-1 | To do |
| 5 | Before | Microsoft app registration | Redirect URI `https://bgosafety.refineddata.com/auth/sso/callback`; copy tenant ID, client ID and the client secret value (section 3) | Azure portal → Entra ID | To do |
| 6 | Before | Fill `server/samconfig.toml` | Certificate ARN, tenant ID, client ID (the other values are already filled in) | Your computer | To do |
| 7 | **Deploy (once)** | `sam build && sam deploy` | Review the changes it lists, confirm with **y** | `server/` | To do |
| 8 | After | SSO client secret | Paste it as `ssoClientSecret` (works within 5 minutes, no redeploy) | Secrets Manager → `bgo-safety/prod/app` | To do |
| 9 | After | DNS record | CNAME `bgosafety` → the `ApiDomainTarget` output | GoDaddy → DNS (section 4) | To do |
| 10 | After | Property and user lists | `import-lists.mjs` on the client's spreadsheets; check `data/admins.json` | `server/` | To do |
| 11 | After | Property locations | Nothing to run: Property Managers save each location on site in the app (section 6) | The app | To do |
| 12 | After | Load data | Automatic: the API loads the bundled lists on its first request after the deploy. Run `load-data.mjs` only for later updates | `server/` | To do |
| 13 | After | App and APK | Nothing to set: the app ships pointing at `https://bgosafety.refineddata.com`. Build with `eas build -p android --profile preview` | Your computer (section 7) | To do |
| 14 | After | Error alarm email | Subscribe an email to `bgo-safety-prod-api-errors` | CloudWatch → Alarms | To do |

To use Okta later instead of Microsoft: in `samconfig.toml` set `SsoMethod=okta`, add `SsoDomain=https://bgo.okta.com` and the Okta client ID, deploy again, add the same redirect URI in Okta, and paste the Okta secret.

# Prerequisites

You need one AWS account, a person with permission to deploy, and four tools on the computer you deploy from.

| Item | Version / detail | Check with |
|---|---|---|
| AWS account | A dedicated account (or one with an agreed naming prefix) | AWS console sign-in |
| Deployer access | An IAM Identity Center user or role allowed to create CloudFormation stacks, Lambda, API Gateway, DynamoDB, S3, IAM roles, Secrets Manager, SES and SNS. For a first deploy, AdministratorAccess on a sandbox account is simplest. | `aws sts get-caller-identity` |
| AWS CLI | v2 | `aws --version` |
| AWS SAM CLI | 1.100 or later | `sam --version` |
| Node.js | 22 LTS (the Lambda runs Node.js 22; AWS retired Node.js 20 in April 2026) | `node --version` |
| Okta admin (or Microsoft Entra ID admin) | Only for SSO: someone who can edit the Okta app integration | Okta admin console |

Sign in the CLI once before deploying: `aws configure sso` (Identity Center) or `aws configure` (access keys for a deploy user). All commands in this guide assume the region **ca-central-1** (Canada Central); pass `--region ca-central-1` or set `AWS_REGION`.

The server code is in the `server/` folder of the project zip. Run `npm install` there once; the deploy packages the code for you.

# 1. Amazon SES (email access codes)

Codes are emailed from **no-reply@refineddata.com** through your existing SES setup in us-east-1, while the API and data stay in ca-central-1. The deploy setting SesRegion (default us-east-1) tells the API where to send from. Confirm the items below in us-east-1 rather than starting over.

1. **Verify the domain, not just the address.** SES console → **Configuration → Identities** → check that `refineddata.com` shows *Verified*, with DKIM *Successful*. If only the address is verified, create a domain identity with **Easy DKIM (RSA 2048)** and add the 3 CNAME records it lists to the refineddata.com DNS.
2. **Same region as SesRegion.** SES identities and production access are per region. The sender must be verified, and out of the sandbox, in the region set as `SesRegion` (**us-east-1** for you). It doesn't matter that the API runs in ca-central-1. The identity must be in the same AWS account as the stack.
3. **Leave the sandbox.** SES console → **Account dashboard**. If it says *Sandbox*, codes only reach addresses you verified one by one. Choose **Request production access**, use case *Transactional*, and describe it as one-time sign-in codes to registered users of an internal app. Approval usually takes about a day.
4. **Mail-from domain (recommended).** Set a custom MAIL FROM such as `mail.refineddata.com` so SPF aligns and fewer codes land in spam.
5. **DMARC (recommended).** Publish a DMARC record for refineddata.com if there isn't one.
6. **Configuration set (optional).** To track deliveries and bounces, create one (e.g. `bgo-safety`) and pass its name as the `SesConfigurationSet` deploy parameter.

The Lambda gets permission to send only from identities in this account and region; no SES keys are stored anywhere.

## 2. Amazon SNS (text access codes)

Codes go by SMS through SNS with the *Transactional* message type, sent from the region set as `SnsRegion` (default **us-east-1**, where you already send codes). SMS settings, spend limits and origination numbers are per region, so use your existing us-east-1 ones; nothing is needed in ca-central-1. Mainly confirm the items below in us-east-1.

1. **Leave the SMS sandbox.** SNS console → **Mobile → Text messaging (SMS)**. In the sandbox, texts only reach phone numbers you verify there. Choose **Exit SMS sandbox** and open the support case it links to.
2. **Raise the monthly spend limit.** New accounts start at US$1 a month, which stops sending after roughly a hundred texts. Set **Account spend limit** to a safe ceiling (e.g. US$50) and request a higher quota in the same support case if needed.

3. **Send like the existing 2FA.** Texts go out with the sender ID `RefinedRisk` (`SnsSenderId`) and **no forced origination number**, exactly as Refined Risk's existing 2FA does. Canadian phones show a number rather than the name; SNS picks the route. `Set SnsOriginationNumber=none`: in testing, forcing the +1855 toll-free number got every text to a Telus line blocked as spam, whatever the wording, while the same texts sent this way arrived. The wording is chosen with `SmsWording`: `bgo` (BGO Safety's own text) or `registered-sample`. List both settings explicitly in your samconfig file, because `sam deploy` keeps a setting's previous value when it isn't listed.
4. **Default message type.** Set it to *Transactional* (codes are time-critical). The API also sets this on every message.
5. **Delivery logs (recommended).** Turn on SMS delivery status logging so failed texts show up in CloudWatch.

Numbers are stored as `999-999-9999` and sent as `+1…`; only Canadian and US numbers are supported today.

## 3. Single sign-on: Okta (or Microsoft Entra ID)

Sign-in runs through the API, so your existing Okta integration and its client secret work as they are; the secret never reaches the phone. Microsoft Entra ID works the same way with a client secret or a certificate.

1. The app opens the Okta sign-in page through `<ApiUrl>/auth/sso/start`.
2. Okta sends the user back to the API at `<ApiUrl>/auth/sso/callback`.
3. The API swaps Okta's answer for the user's identity, using the client secret from Secrets Manager.
4. The API shows a short "Signed in" page that opens the app with a one-time pass (valid 2 minutes), and the app exchanges it for a normal sign-in. The app also checks with the API every 2 seconds while sign-in is open, so it signs in and closes the window by itself even when the browser can't hand control back (emulators such as LDPlayer, some in-app browsers).

The person must still be an Active user in BGO Safety; Okta only proves who they are. Roles come from BGO Safety, not from Okta.

**Okta (your current setup)**

1. Okta admin console → **Applications** → your existing app integration.

2. **Sign-in redirect URIs** → add the `SsoRedirectUri` value, `https://bgosafety.refineddata.com/auth/sso/callback`.
3. Check **Grant type** includes *Authorization Code*, and that BGO users are **assigned** to the app.
4. Rotate the client secret (it appeared in a screenshot) and put the new one in Secrets Manager (section 5).

**Microsoft Entra ID (alternative)**

Use this to test single sign-on without Okta access, with your Refined Data Microsoft 365 accounts.

1. Azure portal → **Microsoft Entra ID → App registrations → New registration**. Name *BGO Safety*. Supported account types: *Accounts in this organizational directory only*.
2. Redirect URI: platform **Web**, value `https://bgosafety.refineddata.com/auth/sso/callback`. It's known before deploying, so do this first. Register.
3. From **Overview**, copy the **Application (client) ID** and **Directory (tenant) ID**.
4. **Certificates & secrets → New client secret** (up to 24 months). Copy the secret **Value** right away (not the Secret ID); it's shown only once. Alternatively upload a certificate and use `ssoConfigType: certificate` (section 5).
5. **API permissions:** Microsoft Graph delegated `openid`, `profile`, `email` (User.Read is there by default). Grant admin consent if your tenant requires it.
6. Optional: **Token configuration → Add optional claim → ID token → email**. Not required: the API also reads the sign-in name (`preferred_username`), which is normally the work email.
7. Deploy with `SsoMethod=entra`, `SsoTenantId=<Directory ID>`, `SsoClientId=<Application ID>` (leave `SsoDomain` blank), and put the secret value in Secrets Manager as `ssoClientSecret` with `ssoConfigType: clientSecret`.

Who can test: the Microsoft account's email (its sign-in name) must match an Active user in BGO Safety exactly. In the current user list that's william.ren@refineddata.com, carl.paulse@refineddata.com and hugh@refineddata.com (Admins). Plus-addressed test accounts such as caroline+pm@refineddata.com aren't real Microsoft accounts, so they can only sign in with access codes. To test as a Property Manager, add a real Microsoft account to the user list with a property assignment.

Switching to Okta later only changes the deploy settings and the secret; the app doesn't change, and its button label follows the provider the API reports.

| Setting | Okta | Microsoft | Where it goes |
|---|---|---|---|
| Provider | `okta` | `entra` | Deploy parameter `SsoMethod` |
| Domain / tenant | `https://bgo.okta.com` | Directory (tenant) ID | `SsoDomain` / `SsoTenantId` |
| Client Id | from the app integration | Application (client) ID | `SsoClientId` |
| Custom authorization server | only if used, e.g. `https://bgo.okta.com/oauth2/default` | not needed | `SsoIssuer` (optional) |
| Client secret or certificate | your secret | secret or certificate | Secrets Manager, section 5 |
| Allowed email domains | optional | optional | `AllowedEmailDomains`, e.g. `bgo.com,refineddata.com` |

The app's **Sign in with Okta** button is always shown. Until these settings are deployed, tapping it says single sign-on isn't set up yet. Once deployed, the button uses the label the API reports (`SsoButtonLabel`, default "Sign in with Okta" or "Sign in with Microsoft").

## 4. Deploy the stack

One SAM template (`server/template.yaml`) creates everything, and `server/samconfig.toml` holds all the deploy settings, so the deploy is a single command with no questions. The first deploy takes about 5 minutes.

1. Open `server/samconfig.toml` and replace the three `<...>` values: `ApiCertificateArn`, `SsoTenantId`, `SsoClientId`. Everything else is filled in for your setup (us-east-1 SES and SNS, `bgosafety.refineddata.com`, Microsoft SSO, demo mode on). A placeholder left in `ApiCertificateArn` stops the deploy before anything is created.
2. Run:

```
cd server
npm install
npm test                    # checks of the API logic, no AWS needed
sam build
sam deploy
```

SAM lists everything it will create and asks **Deploy this changeset? [y/N]**: answer **y**. Re-running the same two commands later updates the stack in place.

The settings available in `samconfig.toml` (`parameter_overrides`):

| Parameter | Default | Set to |
|---|---|---|
| `Stage` | `prod` | `prod`, or `test` for a second copy |
| `SesFromAddress` | `no-reply@refineddata.com` | Your verified SES sender. Two related settings, SesRegion and SnsRegion, default to us-east-1: the regions where your SES sender and SNS texting are set up |
| `SesFromName` | `BGO Safety` | Name shown in the inbox |
| `SesConfigurationSet` | blank | Optional, from section 1 |
| `SnsOriginationNumber` | blank | none (section 2): forcing the toll-free number got texts blocked by Telus. List it as SnsOriginationNumber=none; SAM rejects an empty value and keeps an old value when the setting isn't listed |
| `SnsSenderId` | blank | Default RefinedRisk, as the existing 2FA. List it explicitly (SnsSenderId=RefinedRisk): sam deploy keeps a setting's previous value when it isn't listed. Text wording: SmsWording=bgo ("Your BGO Safety verification code is 123456. It expires in 60 seconds.") or registered-sample ("RefinedRisk - Verification code is 123456"); the texts themselves are in server/src/notify.mjs |
| `SsoMethod` | blank (SSO off) | `okta`, or `entra` for Microsoft |
| `SsoDomain` | blank | Okta: `https://bgo.okta.com` |

| Parameter | Default | Set to |
|---|---|---|
| `SsoTenantId` | blank | Microsoft: Directory (tenant) ID |
| `SsoClientId` | blank | Client Id from Okta / Microsoft |
| `SsoIssuer` | blank | Only for an Okta custom authorization server |
| `SsoButtonLabel` | blank | Optional button text |
| `AllowedEmailDomains` | blank | Optional, e.g. `bgo.com,refineddata.com` |
| `DemoMode` | `true` | `false` once real surveys start (stops generated 2024–2025 demo surveys) |
| `CorsAllowOrigin` | `*` | Only matters for the browser preview |
| `SessionDays` | `30` | Days before users must sign in again |

What gets created:

| Resource | Name | Notes |
|---|---|---|
| HTTP API | `bgo-safety-prod` stage | Throttled to 20 requests/s, bursts of 50 |
| Lambda | `bgo-safety-prod-api` | Node.js 22, arm64, 1024 MB, 30 s timeout (room for PDF reports), X-Ray tracing |
| DynamoDB table | `bgo-safety-prod` | On-demand, encrypted, point-in-time recovery, kept if the stack is deleted |
| S3 bucket | `bgo-safety-prod-<account id>` | Private, encrypted, versioned, HTTPS only, kept if the stack is deleted |
| Secret | `bgo-safety/prod/app` | Generated session signing key + SSO fields (section 5) |
| Log group + alarm | `/aws/lambda/bgo-safety-prod-api` | 90-day logs; alarm at 5+ errors in 5 minutes |

The deploy prints these outputs; keep them: **ApiUrl** (goes in the app), SsoRedirectUri (goes in Okta or Microsoft, section 3), **TableName** (for loading data), **BucketName**, **SecretArn**, **FunctionName**. Check it's up with `curl <ApiUrl>/health`.

## Custom domain (optional): bgosafety.refineddata.com

The API's permanent address is `https://bgosafety.refineddata.com` (no `/prod`), set up in the same single deploy:

1. **Before the deploy:** put your `*.refineddata.com` certificate's ARN (in ca-central-1, see below) in `samconfig.toml` as `ApiCertificateArn`. `ApiDomainName` is already `bgosafety.refineddata.com`.
2. **After the deploy:** add the GoDaddy CNAME record below, pointing `bgosafety` at the `ApiDomainTarget` output. That's the only value that exists only after deploying, and DNS needs no redeploy.
3. **Check:** `curl https://bgosafety.refineddata.com/health` (DNS can take a few minutes).

Because the address is fixed in advance, the Microsoft redirect URI (`https://bgosafety.refineddata.com/auth/sso/callback`) and the app's `API_BASE_URL` can both be set before deploying.

**Already have a wildcard certificate?** A `*.refineddata.com` certificate covers `bgosafety.refineddata.com`, so you can use its ARN as `ApiCertificateArn` and skip requesting a new one (and the validation record below). It must be in **ca-central-1**: check with `aws acm list-certificates --region ca-central-1 --includes keyTypes=RSA_2048,EC_prime256v1`. If it's only in another region (often us-east-1, for CloudFront), it can't be used by the API: either import the same certificate into ca-central-1 (for a certificate bought elsewhere; you need its private key, and you renew it by hand), or request a new one in ca-central-1 (for an AWS-issued one; it usually validates at once because the existing validation record still applies). A wildcard covers one level only: not `refineddata.com` itself or `x.bgosafety.refineddata.com`.

**In GoDaddy** (refineddata.com → **DNS → DNS Records → Add New Record**, type **CNAME**). GoDaddy adds `.refineddata.com` to the Name, so enter only the part before it:

| Record | Name (GoDaddy "Name") | Value (GoDaddy "Value") | When |
|---|---|---|---|
| Certificate validation | ACM's record name minus `.refineddata.com.`, e.g. `_3f1a2b….bgosafety` | ACM's record value, e.g. `_9c8d7e….xyz.acm-validations.aws` | Step 1, before the deploy. Leave it in place: ACM uses it to renew the certificate automatically |
| API address | `bgosafety` | The `ApiDomainTarget` output, e.g. `d-abc123.execute-api.ca-central-1.amazonaws.com` | Step 3, after the deploy |

TTL: the default (1 hour) is fine; 600 seconds makes testing changes quicker. Check that refineddata.com's nameservers are GoDaddy's (**DNS → Nameservers**); if they point elsewhere, add the records there instead. If the domain has CAA records, one must allow `amazon.com` or the certificate won't be issued. Verify with `nslookup -type=CNAME bgosafety.refineddata.com`.

The AWS address (`https://<id>.execute-api.ca-central-1.amazonaws.com/prod`) also keeps working, which is useful if DNS isn't ready yet.

## 5. Secrets Manager

The deploy creates one secret, `bgo-safety/prod/app`, holding a generated 64-character session signing key. Leave `jwtSecret` alone; changing it signs everyone out.

```
{
  "jwtSecret": "<generated>",
  "ssoConfigType": "clientSecret",
  "ssoClientSecret": "",
  "ssoPrivateCertificate": "",
  "ssoPublicCertificate": "",
  "ssoKeyId": ""
}
```

The SSO fields are only needed if the server will call Microsoft itself (for example Microsoft Graph to read group membership); sign-in works without them. To fill them: Secrets Manager console → the secret → **Retrieve secret value → Edit → Plaintext**.

| `ssoConfigType` | Fill in | In Entra (app registration → Certificates & secrets) |
|---|---|---|
| `clientSecret` | `ssoClientSecret` | Create a client secret; note its expiry date |
| `certificate` | `ssoPrivateCertificate` and `ssoPublicCertificate` as PEM, line breaks written as `\n` | Upload the **public** certificate only |

The API re-reads the secret every 5 minutes, so a value you paste after the deploy (such as the SSO client secret) takes effect within 5 minutes without redeploying. Only the Lambda's role can read the secret.

## 6. Load users, properties and assignments

The table starts empty. The loader writes the users (with their property assignments) and the properties from `server/data/` to DynamoDB with your AWS CLI credentials. For the POC these files already hold BGO's data:

| File | Contents | Today |
|---|---|---|
| `properties.json` | Number, name, address, city, province, postal code, region, type, latitude, longitude | 611 Canadian properties from `property-list.xlsx` (US properties and 39 entries with no usable address removed) |
| `users.json` | Name, email, mobile, role (Admin / Property Manager), title, status, assigned property numbers | 71 users: 6 Admins and 65 Property Managers |

A Property Manager is assigned to every property that lists their email under *Property Manager Email*. Emails not in the user list are ignored, and users with no Canadian property were left out. 542 properties have at least one Property Manager; the rest are visible to Admins only.

**From the client's spreadsheets.** When the client sends updated lists (same columns as `property-list.xlsx` and `BGO_Users.xlsx`), turn them into the two files first:

```
cd server
npm install
node scripts/import-lists.mjs --properties property-list.xlsx --users
users.xlsx
```

It keeps Canadian properties with a street address, assigns each user to the properties listing their email under *Property Manager Email*, leaves out users with no property, makes the people in `data/admins.json` Admins, and adds your Refined Data accounts from `data/extra-users.json`. Locations already saved are kept for unchanged addresses. Passwords or other sign-in columns in the user list are never read.

**Property locations are captured on site, not looked up.** The lists don't need coordinates. The first time a Property Manager (or Admin) opens a property that has no location, the dashboard shows *This property has no location yet*; they confirm they're on site and the phone's GPS position is saved for the whole team, with who saved it and when. After that the app opens that property automatically when someone arrives. **Update location** replaces it if it was saved from the wrong spot. Positions less precise than ±150 m are refused (the app asks them to move outside and try again). It works without signal and uploads with the other changes. Re-importing the property list keeps saved locations unless a property's address changes.

**Loading: the first time is automatic.** The lists in `server/data` are bundled with the API, and its first request after the deploy loads them (once per stack; the CloudWatch log shows `lists_loaded`). Anyone who signed in before that through first-user setup is merged with their entry by email. For later updates, edit the files and run:

```
node scripts/load-data.mjs --table bgo-safety-prod --dry-run   # checks only
AWS_REGION=ca-central-1 node scripts/load-data.mjs --table bgo-safety-prod
```

The dry run stops on problems: a user with no email or mobile, the same email or mobile on two users, or an assignment to an unknown property.

**Re-importing.** Safe to re-run. The files win for property teams: an Admin's team changes in the app are replaced. Users no longer in the file are deactivated and properties no longer in it are hidden; their past surveys and reports are kept. The app wins for roles (someone an Admin promoted stays Admin) and each user's last visited property.

**Access rules.** Admins see every property and are the only ones who can change a property team or make someone an Admin. Property Managers see only their assigned properties. A user whose status isn't `Active` is blocked on their next request.

**Demo history.** While `DemoMode` is `true`, the first time anyone opens a property's surveys the app adds approved 2024 and 2025 demo surveys for that property, so copying last year's answers can be tried. They're flagged `demo: true`. Set `DemoMode` to `false` (and `DEMO_HISTORY = false` in the app) before real use.

## 7. Connect the app

The app ships already set up for the deployed API, so there's nothing to change. For an on-device demo with no server, set `AUTH_MODE = 'local'` instead. These are the two lines in `src/config.js`:

```
export const AUTH_MODE = 'aws';
export const API_BASE_URL = 'https://bgosafety.refineddata.com';
```

In this mode access codes arrive by real email (SES) and text (SNS); the on-screen code banner and the "Prototype" note disappear, and Sign in with Okta works once section 3 is deployed. Surveys, answers, incident reports and photos are stored in AWS, so colleagues see each other's work.

**Test from the command line first:**

```
curl <ApiUrl>/health
curl -X POST <ApiUrl>/auth/request-code -H 'content-type: application/json' -
d '{"identifier":"william.ren@refineddata.com"}'
```

**On Windows**, Command Prompt doesn't treat single quotes as quotes, so use double quotes and escape the inner ones:

```
curl https://bgosafety.refineddata.com/health
curl -X POST https://bgosafety.refineddata.com/auth/request-code -H "content-
type: application/json" -d "{\"identifier\":\"william.ren@refineddata.com\"}"
```

In PowerShell: `Invoke-RestMethod -Method Post -Uri https://bgosafety.refineddata.com/auth/request-code -ContentType 'application/json' -Body '{"identifier":"william.ren@refineddata.com"}'`. An `invalid_json` answer or *Could not resolve host: application* means the quotes were mangled, not that the API is down.

The second call should return `{"ok":true,"ttlSeconds":60,…}` and an email should arrive within seconds. An unregistered address returns the same answer but sends nothing, by design.

**Then in Expo Go or the APK:** restart `npx expo start` (or rebuild the APK) after editing `config.js`, and sign in.

**Without AWS (local API).** `npm run dev` in `server/` runs the same API on your PC with data in memory. Codes print in that terminal instead of being sent. Set `API_BASE_URL` to the address it prints, e.g. `http://192.168.1.20:3000` (your PC's IP, so LDPlayer or a phone on the same Wi-Fi can reach it). Data resets when you stop it.

**Saving surveys.** A survey saves as a whole about half a second after each change. Two people rarely work on the same survey, so there's no live merging. If someone else saved it after you opened it, your next save is held and the app asks: **Use their version** or **Keep mine (replace theirs)**. With no signal, changes stay in the app and are sent with the next change once back online; closing the app while offline loses unsent changes.

**Working offline.** Property Managers often have no signal on site. Surveys and incident reports save on the phone and can be completed and submitted offline; photos are copied into the app's own storage so they can't be lost. A bar under the top bar shows how many changes are waiting, with **Sync now**. Uploads happen automatically when the connection returns (checked every 30 seconds and whenever the app reopens). Properties, teams and surveys already opened stay available offline. Signing in needs a connection.

**PDF reports.** When a survey is submitted (or uploaded after an offline submit), the API builds a PDF report: property, who and when, summary, corrective actions with due dates, every answer with details and notes, up to 3 photos per question, and the signatures. It's stored in S3 at `properties/<property>/reports/<checklist id>-<year>.pdf` and opened from the survey's **PDF report** button. After changing a checklist, run `npm run sync-checklists` in `server/` and redeploy so reports use the new wording.

**Sign-in token.** Kept in the phone's secure storage (Android Keystore / iOS Keychain), not in ordinary app storage.

## 8. Reference: data and API

**DynamoDB, one table** ( `pk` + `sk`, both strings; `ttl` expires temporary items):

| pk | sk | Holds |
|---|---|---|
| `USERS` | `USER#<id>` | User profile, role (Admin / Property Manager), status, assigned property numbers |
| `LOGIN#email#<email>` / `LOGIN#phone#<phone>` | `LOGIN` | Sign-in lookup → user id |
| `PROPERTIES` | `PROP#<number>` | Property: name, address, province, region, type, coordinates |
| `CODE#<email or phone>` | `CODE` | Pending access code (hashed), attempts; expires via `ttl` |
| `RATE#<email or phone>` | `<15-min window>` | Code-request counter; expires via `ttl` |
| `TEMP#state#<id>` / `TEMP#ticket#<id>` | `TEMP` | SSO sign-in in progress (10 min) / one-time pass (2 min) |
| `PROP#<number>` | `SURVEY#<checklist id>#<year>` | Survey with all its answers, status, saved by/at, version |
| `PROP#<number>` | `INC#<id>` | Incident report: form, values, status, number, version |
| `PROP#<number>` | `COUNTER#ACC\|NM#<year>` | Report numbering |

**S3:** photos at `properties/<number>/photos/<uuid>.jpg`. The app gets 5-minute upload links and 1-hour view links from the API, only for properties the user can access.

**S3 reports:** PDF reports at `properties/<number>/reports/<checklist id>-<year>.pdf`, opened through 1-hour links for users who can access the property.

**API** (all JSON; every route except `/auth/*` and `/health` needs `Authorization: Bearer <session token>`):

| Method | Path | Purpose |
|---|---|---|
| POST | `/auth/request-code` | Send a code (same answer for unknown addresses) |
| POST | `/auth/verify-code` | Check a code → session token |
| POST | `/auth/complete-setup` | First user only, when the table has no users |
| GET | `/auth/sso/status` | Is SSO configured, and the button label |
| GET | `/auth/sso/start?redirect=` | Starts Okta / Microsoft sign-in (browser). The app uses POST /auth/sso/begin, which also returns a private poll id, and GET /auth/sso/poll?id= to collect the one-time pass |
| GET | `/auth/sso/callback` | Where Okta / Microsoft return; swaps the code using the secret |
| POST | `/auth/sso/redeem` | One-time pass → session token |
| GET / PATCH | `/me` | Signed-in user; update last property, notification flags. Own mobile number: POST /me/phone/request then /me/phone/verify (code texted to the new number), DELETE /me/phone removes it |
| GET | `/properties` | Properties this user can access |
| GET / PUT | `/properties/{n}/team` | Who is assigned / set the team (Admins) |
| GET | `/users` | Active users, for team editing (Admins) |
| PATCH | `/users/{id}/role` | Make Admin or Property Manager (Admins; not yourself) |
| GET / POST | `/properties/{n}/surveys` | List / start one (copied answers allowed) |
| GET / PUT / DELETE | `/surveys/{id}` | Read / save with `baseVersion` (409 + their copy if outdated; `force` overrides) / discard a draft |

| Method | Path | Purpose |
|---|---|---|
| POST | `/surveys/{id}/submit` | Save and submit; builds the PDF report (GET /surveys/{id}/pdf returns a link to it) |
| POST | `/import-surveys` | Past surveys: Admins any property; anyone for their own properties in DemoMode |
| GET / POST | `/properties/{n}/incidents` | List visible reports / start a draft |
| PUT / DELETE | `/incidents/{n}/{id}` | Update / delete own draft |
| POST | `/incidents/{n}/{id}/submit` | Submit; server assigns the number |
| POST / GET | `/uploads`, `/uploads/url?key=` | Photo upload link / view link |

Incident visibility: drafts only to their author; submitted accident records to the author and Admins; submitted near misses to everyone on the property.

## 9. Operate: security, monitoring, costs, troubleshooting

**Security checklist before real users**

- [ ] SES and SNS out of their sandboxes; SNS spend limit set
- [ ] Property and user lists reviewed; demo mode turned off
- [ ] `CorsAllowOrigin` set to the preview's address (or left `*` only while testing)
- [ ] Deploy credentials are short-lived (IAM Identity Center), not long-lived access keys
- [ ] AWS Budgets alert on the account (e.g. US$25/month)
- [ ] If SSO is on: `AllowedEmailDomains` set; client secret expiry in a calendar
- [ ] Privacy review of incident reports (health information): who may read accident records, retention period

Already built in: codes hashed and never returned; same answer for unknown addresses; 3 attempts, 60-second expiry, 5 requests per 15 minutes per address; API throttling; all data encrypted at rest; HTTPS only; table and bucket kept if the stack is deleted; point-in-time recovery and S3 versioning for undo.

**Monitoring.** Logs: CloudWatch → `/aws/lambda/bgo-safety-prod-api` (90 days). Unhandled errors log as `unhandled_error`; codes requested for unknown addresses log as `code_request_unknown_identifier`. The alarm `bgo-safety-prod-api-errors` fires at 5+ errors in 5 minutes; subscribe an email to it via an SNS topic. X-Ray shows slow calls.

**Costs (estimate, POC scale: ~150 users).** Lambda, API Gateway, DynamoDB on-demand, S3 and Secrets Manager together: a few US dollars a month, much of it in the free tier. Texts are the main variable: around a cent each to Canadian numbers (check current SNS pricing). Emails: about US$0.10 per thousand.

### Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| No email arrives | SES still in sandbox, or sender not verified in the SesRegion (us-east-1) | Section 1; check the Lambda log for `MessageRejected` |
| No text arrives | SMS sandbox, spend limit reached, or number not Canadian/US | Section 2 |
| "Too many codes requested" | 5 requests in 15 minutes | Wait 15 minutes |
| App says it can't reach the sign-in service | Wrong `API_BASE_URL`, or the phone can't reach it | `curl <ApiUrl>/health` from the phone's network |
| APK can't reach the local dev API | Android release builds block plain `http://` | Use Expo Go for local testing, or the deployed HTTPS API |
| User signs in but sees no properties | Property Manager with no assigned properties | An Admin adds them in Property team, or add their email to the property list and re-run the loader |
| SSO says "isn't set up for BGO Safety" | Okta / Microsoft email not in the user list, inactive, or domain not allowed | Add or activate the user; check `AllowedEmailDomains` |
| Everyone signed out at once | `jwtSecret` changed | Expected; sign in again |

**Updating.** Code or template changes: `sam build && sam deploy`. User or property changes: edit `server/data/` and re-run the loader.

**Removing it.** `sam delete --stack-name bgo-safety-prod` removes the API, Lambda, secret and alarm. The table and bucket are kept on purpose (they hold the data); delete them separately in the console when you're sure.